FRISBEE — The Math

One idea at a time, in the order the system uses them

Step 1 — Turning words into numbers (TF-IDF)

Each daily message is a bag of words. We want to know how much of each of the four clusters — *drift*, *align*, *excuse*, *energy* — is present. For every cluster c, the score is:

$$x_c = \sum_{w \in c \cap \text{message}} \text{TF}(w) \cdot \text{IDF}(w)$$

TF (term frequency) counts how often word w appears in today's message, normalised by message length. **IDF** (inverse document frequency) rewards words you use rarely but used today:

$$\text{IDF}(w) = \log\left(\frac{N+1}{df(w)+1}\right)$$

where N is the number of messages so far and $df(w)$ is how many of those messages contained w. A word you use every day has high df, low IDF, and therefore low weight — it tells us nothing new. A word you almost never use, but used today, has low df, high IDF, and carries real signal.

The cold-start fix: blended IDF

In week one, $N = 1$ and $df(w) = 1$ for every word in the message, so $\text{IDF}_{\text{personal}} = \log(1) = 0$ for everything. The feature vector collapses to zero and the model cannot learn. The fix is to blend with a background IDF pre-computed from a large corpus (stored in `vocab.json`):

$$\text{IDF}_{\text{blend}}(w) = \alpha \cdot \text{IDF}_{\text{personal}}(w) + (1-\alpha) \cdot \text{IDF}_{\text{background}}(w)$$

As weeks accumulate, α can grow toward 1, making the model progressively more personal and less generic.

The result of Step 1 is a four-dimensional feature vector:

$$\mathbf{x} = [x_{\text{drift}},\ x_{\text{align}},\ x_{\text{excuse}},\ x_{\text{energy}}] \in \mathbb{R}^4$$

Step 2 — Predicting the week (logistic regression)

We have weights $\mathbf{w} \in \mathbb{R}^4$ and a bias $b \in \mathbb{R}$, both initialised to zero. The prediction is:

$$z = \mathbf{w}^\top \mathbf{x} + b = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

z can be any real number. We squash it into a probability using the sigmoid function:

$$\hat{p} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$\hat{p} \in (0,1)$ is the model's estimate of the probability that this is a "yes-week." A binary prediction is obtained by thresholding at 0.5.

Step 3 — Measuring how wrong we were (loss)

At the end of the week, the user provides the ground truth $y \in \{0, 1\}$. The loss is binary cross-entropy:

$$\mathcal{L} = -\left[y \log \hat{p} + (1 - y) \log(1 - \hat{p})\right]$$

When $y = 1$ and $\hat{p} \approx 1$, the loss is near zero. When $y = 1$ and $\hat{p} \approx 0$, the loss is large. The loss is never negative.

Step 4 — Updating the weights (gradient descent)

We want to reduce $\mathcal{L}$ by adjusting $\mathbf{w}$ and b. The gradient of $\mathcal{L}$ with respect to $\mathbf{w}$ is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (\hat{p} - y)\,\mathbf{x}$$

This is one of the elegant results of logistic regression: the gradient is simply the prediction error times the input. With L2 regularization added to the weights (but not the bias, to preserve the user's base rate):

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} - \eta\left[(\hat{p} - y)\,\mathbf{x} + \lambda\,\mathbf{w}\right]}$$

$$b \leftarrow b - \eta\,(\hat{p} - y)$$

with learning rate $\eta = 0.1$ and regularization strength $\lambda = 0.01$.

Why L2 on w only? Regularizing b would pull predictions toward $\hat{p} = 0.5$ regardless of evidence. The bias captures the user's actual base rate of yes-weeks; penalising it destroys that signal. Regularizing $\mathbf{w}$ prevents any single cluster from dominating when data is sparse.

What the model learns over time

After T weekly updates, the weight w_c for cluster c reflects how predictive that cluster's language has been for *this user*. If your drift words reliably precede no-weeks, w_{drift} becomes strongly negative. If your energy words are noise for you personally, w_{energy} stays near zero. No other user's data influences these weights. That is the entire point.

Symbol	Meaning	Value
$\mathbf{x} \in \mathbb{R}^4$	cluster-weighted TF-IDF feature vector	computed daily
$\mathbf{w} \in \mathbb{R}^4$	learned weights	init $\mathbf{0}$, updated weekly
$b \in \mathbb{R}$	bias (base rate)	init 0, updated weekly
$\hat{p} \in (0, 1)$	predicted yes-week probability	$\sigma(\mathbf{w}^\top \mathbf{x} + b)$
$y \in \{0, 1\}$	ground truth (user label)	provided weekly
η	learning rate	0.1
λ	L2 regularization strength	0.01

Each update is one line of arithmetic. The model has no hidden layers, no embeddings, no external API calls. Everything runs in the browser.